

SOFTWARE ENGINEERING

Introduction

Abdulfattah Alfarra

OUTLINE

- ▣ **Software**
- ▣ **Professional Vs. Student Software**
- ▣ **Degree of Difficulty for software project**
- ▣ **Software Engineering**
- ▣ **Software Failure**
- ▣ **Build or Buy**
- ▣ **LAWS ON SOFTWARE ENGINEERING**

Software

▣ Software – As IEEE definition

Software is:

- 1) Computer programs (code)
- 2) Procedures
- 3) Documentations
- 4) Data affecting the operation of a computer system.

Software

- ▣ All four components are needed in for software development process and the coming years of maintenance services for the following reasons:
 - 1) **Computer programs** (the “code”) are needed because, obviously, they activate the computer to perform the required applications.

Software

2) **Procedures** are required, to define the order and schedule in which the programs are performed, the method employed, and the person responsible for performing the activities that are necessary for applying the software.

Software

- 3) Various types of **documentation** are needed for developers, users and maintenance personnel. The development documentation (the requirements report, design reports, program descriptions, etc.) allows efficient cooperation and coordination among development team members and efficient reviews and inspections of the design and programming products.
- ▣ The user's documentation (the "user's manual", etc.) provides a description of the available applications and the appropriate method for their use.

Software

- 4) **Data** including parameters, codes and name lists that adapt the software to the needs of the specific user are necessary for operating the software.
- ▣ Another type of essential data is the standard test data,
 - ▣ Also, there are types of should include data such as web-based systems, expert systems and data mining systems.

Efforts of software Engineering Activates

● Project Management	8.08%
● Requirements	14.43%
● Design	11.36%
● Coding	13.50%
● SQA	30.64%
● SCM	13.02%
● Integration	6.54%
● Misc.	~3%

Types of Software

- Software products may be
 - **Generic** - developed to be sold to a range of different customers

e.g. PC software such as Excel or Word.

- **Custom** - developed for a single customer according to their specification.

Customer could be external or Internal

Professional Vs. Student Software

- ▣ A student can build a software system for 2 to 3 months
- ▣ The same system may take a software Engineer to build 10 months why?
- ▣ The answer, of course, is that two different things are being built in the two scenarios.
- ▣ In the first, a student system is being built which is primarily meant for demonstration purposes, and is not expected to be used later. Because it is not to be used, nothing of significance depends on the software and the presence of bugs and lack of quality is not a major concern. Neither are the other quality issues like usability, maintainability, portability etc.

Professional Vs. Student Software

- ▣ On the other hand, an **industrial-strength** software system is built to solve some problem of a client and is used by the client's organization for operating some part of business, and a malfunction of such a system can have huge impact in terms of financial or business loss, inconvenience to users, or loss of property and life.
- ▣ Consequently, the software system needs to be of high quality with respect to properties like reliability, usability, portability, etc.
- ▣ This need for high quality and to satisfy the end users has a major impact on the way software is developed and its cost. The rule of thumb suggests that the industrial-strength software may cost about 10 times the student software

Degree of Difficulty for software project

Characteristics	Low	Moderate	High
Number of functions performed or (LOC)	Small	Medium	Large
Novelty of Functions	Standard	Similar to Existing System	New Theory
Number of Concurrent Access	1	Several	Many
Multitasking	No	Some	Yes
Interactive vs. batch access	Batch	Some	Highly
Response time requirement	Off-line	Moderate	Real-Time

Degree of Difficulty for software project

Characteristics	Low	Moderate	High
Need for distribution	None	2 computers	Many
Amount of data stored	Small	Medium	Large
Structure of Data	Simple data relationship	Moderate	Complex Relationship
Accuracy of Data	Low	Moderate	High
Tolerance to downtime	Can tolerance Several hours	Can tolerance short period	Can tolerance no downtime
Security needs	None	Moderate	High

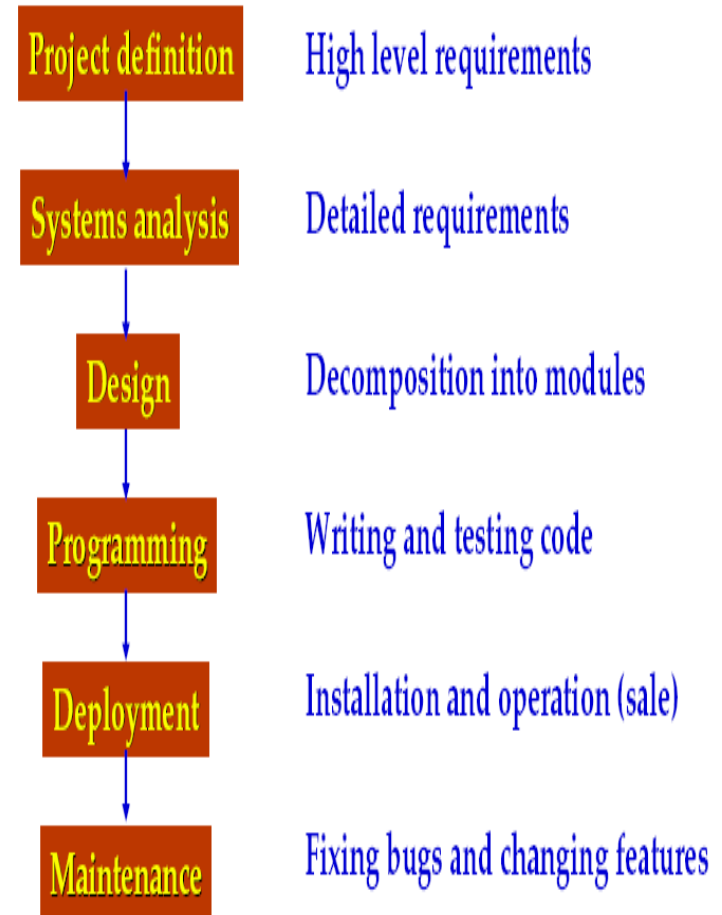
Degree of Difficulty for software project

Characteristics	Low	Moderate	High
Interaction with other systems	None	Some, well defined	Much
Dependence of Hardware	Independent	Some	Tied to specific
Stability of Specifications	Fixed customer requirements	Some changes may occurs	Frequent Change
User Sophistication	Familiar	Familiar with other similar systems	Naive
Developer Sophistication	Has developed similar systems	Experience with tools, not systems	No Experience

Software Engineering

What is software engineering?

- ▣ Software engineering is an engineering discipline that is concerned with all aspects of software production (Not only programming or coding)



Software Engineering

Why Software Engineering?

- ▣ In software engineering we are not dealing with programs that people build to illustrate something or for hobby (which we are referring to as student systems). Instead the problem domain is the software that solves some problem of some users where larger systems or businesses may depend on the software, and where problems in the software can lead to significant direct or indirect loss. We refer to this software as *industrial strength software*.

Software Engineering

- ▣ Software Engineering is a discipline whose aim is the production of quality software, delivered on time, within budget, and satisfying users' needs

Software Engineering

- ▣ Software engineering originally emerged as a response to the software development crisis of the 1960s. It subsequently evolved as an engineering like response to project failures, serious economic losses, scheduling delays, competitive markets, and increasingly demanding customers looking for functionality, quality, and reliability at minimum cost.

Factors of Software Engineering

In software, the four main factors that influence Software Engineering

- ▣ Project ;
- ▣ People ;
- ▣ Process ;
- ▣ Product .

Project

The project management process specifies all activities that need to be done by the project management to ensure that cost and quality objectives are met.

- ▣ Planning,
- ▣ Monitoring and control,

Project : planning

- ▣ During planning, the major activities are determine such as cost estimation, schedule and milestone determination, project staffing.

Project : Monitoring and control

- ▣ It includes all activities the project management has to perform while the development is going on to ensure that project objectives are met and the development proceeds according to the developed plan

Process

- ▣ A **set of activities** whose goal is the development or evolution of software
- ▣ Generic activities in all software processes are:
 - **Specification** - what the system should do and its development constraints
 - **Development** - production of the software system
 - **Validation** - checking that the software is what the customer wants
 - **Evolution** - changing the software in response to changing demands

People

Software Engineering Jobs/Roles

- ▣ **Systems Analyst** -- analyzes requirements for an application,
- ▣ **Software Architect** – designs the overall structure of the application
- ▣ **Software Programmer** – implements the design using software development tools,
- ▣ **Software Systems Administrator** – administers user accounts, technology refreshment, software deployment to users, software problem solvers
- ▣ **Software Database Administrator** – administers the database (installation, maintenance, backup, refreshment)

People

- ▣ **Customer Support Engineer** – solves customer, end-user problems with computer applications, configuration (e.g. ISP)
- ▣ **Webmaster** – designs, implements, and maintains a web site
- ▣ **Software Security Engineer** – identification, authorization, authentication, data protection, data integrity)
- ▣ **Software Tester** -independent verification and validation
- ▣ **Software Project Manager** –plan, organize, direct, coordinate, control a software project (emphasis on risk management)–
- ▣ **Software Quality Manager/Engineer** – software reliability modeling, statistical quality control, defect analysis

Products

The work product is the programs, documents, and data that are computer software deliver.

- ▣ **Requirement Definition:**

- Rapid prototype, or
- Requirements document

- ▣ **Specification**

- Specification document

- ▣ **Design**

- Architectural Design
- Detailed Design

Products

- ▣ **Coding**
 - Source code
 - Test cases
- ▣ **Testing**
 - Integration testing
 - Acceptance testing
- ▣ **Maintenance**
 - Change record
 - Regression test cases

Software Engineering

Software project without Software Engineering	Software Project with Software Engineering
Individual view on software development activities.	Team view on software development activities
A programmer, as software developer, is a master of all skills needed for programming.	A software engineer skills for a single or limited development process(es).
Final products reflect personal talent, art, hobby and experience	Final products are based on sound theory, proven methodologies and common practices.

Software Engineering

Software project without Software Engineering	Software Project with Software Engineering
A software developer is a person who has multi-roles as of requirement analyst, designer, programmer, tester, and even user.	A software engineer is a person who has specific role in one of the processes as listed on the left.
Programmers are not trained in formal ways, but believe learning by doing.	Software engineers are trained in formal ways and following common rules
Programming is personal interaction between the programmer and a computer.	Programming is a group interaction between all roles and processes, including the

Software Engineering

Software project without Software Engineering	Software Project with Software Engineering
Problems to be solved are limited in small scale	Problems to be solved are large-scale software development for complicated systems.
Knowledge transfer is hard in programming, and design and implementation of software is regarded as personal experience.	Knowledge transfer is defined and carried out by hierarchical processes at organization, project, and individual levels.
Program maintenance is relied on the original designer.	Software maintenance can be carried out independently from the original developers

The essential characteristics of Software Engineering (1)

1) **Software Engineering concerns the construction of large applications**

Small applications generally refers to application written by one person in relatively short period.

Large applications refer to multi-person job that span for at least six months.

Tools, techniques and methods needed for small applications are different than for large applications

The essential characteristics of Software Engineering (2)

2)The Central Theme is Mastering Complexity

One is forced to split the problem into parts such that each individual part can be grasped, while the communication between parts remain simple. The total complexity does not decreased in this way ,but it becomes manageable.

The essential characteristics of Software Engineering (3)

3) Regular Cooperation between people in an integral part of large applications

Since the problem are large , many people have to work concurrently at solving these problems. There must be a clear arrangements for the distribution of work, methods of compunctions, responsibilities, and so on.

Arrangements alone is not sufficient, one has to stick to those arrangements. In order to enforce them , standards or procedures supported by tools maybe employed.

The essential characteristics of Software Engineering (4)

4) Software evolves:

Software models part of reality. This reality evolves. For example, banks regulations always changes. So the software should be evolves in order to not become obsolete. The evolutions should be bear in mind during development.

The essential characteristics of Software Engineering (5)

5) The efficiency with which software is developed is of crucial importance

Total cost and the development time of software project is high. This is also for maintenance of software. Important themes within the field of software Engineering concern better and more efficient methods and tools for development and maintenance of software

The essential characteristics of Software Engineering (6)

6) The software has effectively support its users:

Software is developed to support users at work. The functionality offered should fit users' tasks. Users that not satisfied with the system will try to avoid it

Effective user support means that must carefully study users at work in order to determine the proper functional requirements, and that we have to address usability and other quality aspects as well, such as reliability , responsiveness , user-friendliness. It also means that software development entails more than delivering software . User manuals and training material also important.

Software Failure

- 1968: Conference on **'software crisis'**.
 - Delivery of software was sometimes years **late**.
 - Its **cost** was often much higher than predicted.
 - Many programs were unreliable.
 - **Maintenance** of software tended to be difficult.
 - The software often **poorly performed** the task for which it was designed.
- ▣ ⇒ The term **'software engineering'** was coined.

Software Failure

- ▣ A software failure is a software project that has one or more of the following:
 - Over budget
 - Late
 - Does not satisfy user needs or expectations
 - Does not meet functional or performance requirements
 - Does not meet quality requirements

Software Failure

Software crisis:

- ▣ For every 8 large project, 2 are cancelled.
- ▣ Most projects exceed their time-schedule by 50%.
- ▣ 75% of large systems malfunction after completion.
- ▣ Detecting and correcting errors take 50% of time and effort.
- ▣ Maintenance cost of a system is 60% of entire cost.

Software Failure

What is causing software Failure?

- 1) Software requirements do not adequately describe user needs or customer expectations
- 2) Project planning is frequently unrealistic, incomplete, or ignored
- 3) Project cost and schedule estimates are underestimated or established only by management.

Software Failure

- 4) Software quality is difficult to specify, design, and build-to
- 5) Software development progress is difficult to see, progress is often unknown
- 6) Changes in requirements are not accompanied by changes in software plans
- 7) Design is changed without changing requirements
- 8) Standards are not used or documented

Software Failure Example:

Space Shuttle Software

- ▣ Cost: \$10 Billion, millions of dollars more than planned
- ▣ Time: 3 years late
- ▣ Quality: First launch of Columbia was cancelled because of a synchronization problem with the Shuttle's 5 onboard computers.
 - Error was traced back to a change made 2 years earlier when a programmer changed a delay factor in an interrupt handler from 50 to 80 milliseconds.
 - The likelihood of the error was small enough, that the error caused no harm during thousands of hours of testing.

Build or Buy

- ▣ Software is expensive to build — the fixed costs are very high, and the development takes a lot of time and human resources
- ▣ As long as the needs are similar, spreading such costs across multiple organizations should make a lot of sense to both sellers and buyers.
- ▣ We have seen enterprise software implementations increase, especially among the larger companies in the areas of manufacturing, finance, and human resources.

Build or Buy

- ▣ When developing software, research has shown that there is an inverse correlation between the size of a project and the probability that it will actually be completed

Project Size	People	Time (months)	Success
< \$750k	6	6	55%
\$750k-\$1.5M	12	9	33%
\$1.5M-\$3M	25	12	25%
\$3M-\$6M	40	18	15%
\$6M-\$10M	250+	24+	8%
\$10M+	500+	36+	0%

Figure 4.9 *

Build or Buy

- ▣ However, in many other areas of the organization, custom development continues. Why is this so, even though there are so many Customer Off-the-Shelf product (COTS) products available

Build or Buy

- ▣ Custom Software – Advantages:
- ▣ **Gives you exactly what you need** – Each business has its own set of unique business rules. Often, the only way they can be computerized is via custom software.
- ▣ **You own the source code** – Owning the source code affords you more control over future enhancements. When clients see the value of custom software, they become more sophisticated and quickly find specific ways to improve their particular business work flow. When ideas pop into your entrepreneurial brain, you can have the existing software modified to incorporate those ideas. Additionally, owning the custom source code is a valuable asset; an asset that should be included in the price of the business if it is ever sold.

Build or Buy

- ▣ **More useful reports** – There is no reason to have software unless it can produce the desired output; this is often in the form of reports. Custom software allows for the creation of meaningful reports that are used to make smart business decisions. You, the business owner, can ensure that reports give you the information the way you want it; not how the vendor thinks you want it.
- ▣ **More efficient in-house help desk** – Help desk personnel will be more familiar with the business rules involved and can assist users. They will know about common issues, traps, and work-arounds. This is much better than having to explain a specific issue to COTS help desk personnel who usually deal with generic problems

Build or Buy

- ▣ **Decision makers are readily available** – During the design phase, decision makers are available to make judgment calls. They have an intimate knowledge regarding how the software should work. Users are very good at describing the work flow and as a result, the software can be more effectively designed to increase their efficiency.
- ▣ **Users readily accept the software** – If the users have had input into the design of the new software, they more readily accept the changes. Plus, they are ahead of the learning curve because they've been exposed to the software during the development stage.
- ▣ **Money is not wasted on unnecessary features** – A side benefit of getting the exact functionality you want in the software is wisely spending your development dollars. Every dollar that is spent goes toward making the best possible product.

Build or Buy

- ▣ **No major license fees** – You own the code. You own the software. You control its use. This is not to say that annual fees, hardware enhancements, and other “cost of doing business” expenses don’t exist. Most likely, they will exist, but you will need to find out what they cover, how much they are, and compare them to the same fees for the COTS software.

Build or Buy

- ▣ Custom Software – Disadvantages:
 - May cost more** – It is custom software and one would expect to pay more because custom “anything” is labor-intensive. Depending on the scope of the software’s capabilities, it could be considerably more expensive than “canned” software; only you can decide if the derived benefits are worth the expenditure.
 - Not immediately available** - Depending on the number of developers, the scope of the project and other factors, it may take months for the application to be created.

Build or Buy

- ▣ **You may be re-inventing the wheel** – Most applications have many of the same features as requirements; printing reports, displaying data, and adding or modifying data are such examples. Time, money, and effort will have to be spent with respect to these basic tasks.

LAWS ON SOFTWARE ENGINEERING

The following is list of selected laws related to software engineering.

- ▣ A program will always do what you tell it to do, but never what you want it to do.
- ▣ Program complexity grows until it exceeds the capability of the programmer who must maintain it.
- ▣ Every nontrivial program has at least one bug.
- ▣ The smallest bugs cause the greatest damage and problems.

MURPHY'S LAWS ON SOFTWARE ENGINEERING

- ▣ Adding labor to a late software project makes it later.
- ▣ A working program is one that has only unobserved bugs.
- ▣ No matter how much resources you have, it is never enough.
- ▣ A failure in a device will never appear until it has passed final inspection.
- ▣ Computers don't make errors. What they do they do on purpose.